MEMORANDUM

To:	Charlie Refvem, Lecturer, Department of Mechanical Engineering, Cal Poly SLO
	crefvem@calpoly.edu
From:	Quinn Patterson
	qpatters@calpoly.edu
	Jake Rowen
	jhrowen@calpoly.edu
Team:	23
Date:	2/3/26
RE:	**Hardware Driver Creation and Testing**

In lab this week we coded motor and encoder drivers to give general blueprints and useful functions to our hardware. This makes our code overall easier to read and write throughout the quarter. We tested our new drivers to verify the following:

1. Each motor can spin forward and backward independently with varying speed.

2. Each motor can be enabled and disabled individually and does not start moving when enabled.

3. Each encoder position can grow in the positive and negative directions.

4. Timer reload has no effect on the encoder speed or position

5. The encoder and motor objects are paired together properly in code so that spinning a motor causes its associated encoder to count.

6. The encoders each count upward when the motors have a positive duty cycle applied and the wheels rotate such that Romi would drive forward. If this test fails consider whether it is better to fix the sign convention in software or in hardware.

```python
while True:

    if printFLG:
        printFLG = False
        print(f"{Left_Encoder.get_position()} , {Right_Encoder.get_position()}")

    now = ticks_ms()
    dt = ticks_diff(now, state_start)

    if state == 0:
        Left_Motor.enable()
        Right_Motor.enable()
        Left_Motor.set_effort(50)
        Right_Motor.set_effort(50)
        state_start = now
        state = 1

    elif state == 1 and dt >= 1000:
        Left_Motor.set_effort(100)
        state_start = now
        state = 2

    elif state == 2 and dt >= 1000:
        Right_Motor.set_effort(100)
        state_start = now
        state = 3

    elif state == 3 and dt >= 10000:
        Left_Motor.disable()
        Right_Motor.disable()
        Left_Motor.set_effort(50)
        Right_Motor.set_effort(50)
        state_start = now
        state = 4

    elif state == 4 and dt >= 500:
        Left_Motor.enable()
        Right_Motor.enable()
        state_start = now
        state = 5

    elif state == 5 and dt >= 500:
        Left_Encoder.zero()
        Right_Encoder.zero()
        Left_Motor.set_effort(-50)
        Right_Motor.set_effort(-50)
        state_start = now
        state = 6

    elif state == 6 and dt >= 1000:
        Left_Motor.set_effort(-100)
        state_start = now
        state = 7

    elif state == 7 and dt >= 1000:
        Right_Motor.set_effort(-100)
        state_start = now
        state = 8

    elif state == 8 and dt >= 5000:
        Left_Motor.disable()
        Right_Motor.disable()
        state = 8   # done

    sleep(0.01)
```

Figure 1. Testing Code in main.py

Figure 1 above showcases the testing code we used to fulfill our 6 requirements. Throughout the whole time we have a callback running at 200Hz to print our encoder values accurately. We used a FSM structure to allow for printing of the encoders throughout the process. This is superior to a sleep() approach because sleep() blocks other code from running. State 0 enables our motors and sets both motors to 50% effort. We then set left motor effort to 100% in state 1 and right motor to 100% in state 2. Both state 0 and 1 transition after a second and shows that we can modify motor efforts independently. State 2 runs for 10 seconds to show us our encoder values do not get affected by timer reload. In subsequent states we disable the motors, try and run them to show disable works, then reenable to show that enable sets the motor effort to zero. Afterwards we zero the encoders and run the motors in reverse at 50 and then 100% effort to showcase motors can run in reverse and that encoders can count in the negatives. Afterwards we disable both motors.

main.py , encoder_class.py , and motor_class.py are included as attachments and include all our code for the week.